Building and Deploying a Demon Application (Step-by-Step + Internals)

1. Creating the Application Directory

```
mkdir -p app-demo
cd app-demo
```

2. Creating the JavaScript Application

```
vim app.js

const express = require('express');

const app = express();


const PORT = 3000;


app.get('/', (req, res) => {

  res.send('Hello from Kubernetes 🚀 ');

});


app.get('/health', (req, res) => {

  res.status(200).json({ status: 'ok' });

});


app.listen(PORT, () => {

  console.log(` App running on port ${PORT}` );

});
```

What it does

This is a very small Node.js web application.

It defines two HTTP endpoints:

- / returns a text response
- /health returns a JSON health response

Why it was used

The application is intentionally simple because the project focus is on:

- containerization
- orchestration
- CI/CD
- health checks
- service exposure

Under the hood

- express is a lightweight web framework for Node.js
- app.get(...) defines routes that answer HTTP GET requests
- app.listen(PORT) starts the web server on port 3000

Endpoint behavior

- Visiting / returns:
  - Hello from Kubernetes 🚀
- Visiting /health returns:
  - HTTP 200
  - JSON payload: {"status":"ok"}

The /health endpoint is especially important because Kubernetes uses it for readiness and liveness probes.

##3. Creating package.json

vim package.json

{

```
  "name": "app-demo",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

What it does

This file defines the Node.js project metadata and dependencies.

Under the hood

- name and version identify the project
- main points to the main application file
- scripts.start defines how the app is started
- dependencies tells npm which libraries must be installed

Why it matters

When Docker builds the image, it reads this file and installs express automatically.

##4. Creating the Dockerfile

vim Dockerfile

```
FROM node:20-alpine

WORKDIR /app
```

```
COPY package.json ./

RUN npm install


COPY . .


EXPOSE 3000


CMD ["npm", "start"]
```

What it does

This file tells Docker how to build the container image.

Under the hood

- FROM node:20-alpine
  - uses a lightweight Node.js base image
- WORKDIR /app
  - sets /app as the working directory inside the image
- COPY package.json ./
  - copies the dependency definition into the image
- RUN npm install
  - installs required Node.js packages
- COPY . .
  - copies the rest of the application files into the image
- EXPOSE 3000
  - documents that the container listens on port 3000
- CMD ["npm", "start"]
  - defines the default command used when the container starts

Why this order matters

Copying package.json before copying the full source allows Docker to cache dependency installation layers more efficiently.

##5. Building the Docker Image

docker build -t andreiboboc123/app-demo:1.0 .

What it does

Builds a Docker image from the current directory.

Under the hood

Docker reads the Dockerfile line by line and creates image layers:

1. pull base image
2. create working directory
3. copy package file
4. install dependencies
5. copy application source
6. store final image locally

Tag explanation

- andreiboboc123 = Docker Hub username
- app-demo = repository/image name
- 1.0 = image tag/version

Why tagging matters

Tags make image versions traceable and reusable in deployments.

##6. Log in to Docker Hub

docker login

What it does

Authenticates the Docker CLI with Docker Hub.

Under the hood

- Docker stores authentication data locally
- Later docker push requests use these credentials to upload images

Why it matters

Without authentication, Docker Hub will reject image pushes to your repository.

##7. Push the Image to Docker Hub

```
docker push andreiboboc123/app-demo:1.0
```

What it does

Uploads the built image to Docker Hub.

Under the hood

Docker compares local image layers with remote layers and pushes only missing layers.

Why this matters

Kubernetes nodes need a registry they can pull images from. Docker Hub acts as that registry in this project.

##8. Create the Kubernetes Manifest Directory and the necessary manifests

While on the Kubernetes control plane:

```
mkdir -p app_manifests

cd app_manifests

vim deployment.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

  name: app-demo
```

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app-demo
  template:
    metadata:
      labels:
        app: app-demo
    spec:
      containers:
        - name: app-demo
          image: andreiboboc123/app-demo:1.0
          ports:
            - containerPort: 3000
          readinessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 10
```

periodSeconds: 10

What it does

Defines how Kubernetes should run the application.

Under the hood

- The Deployment controller creates a ReplicaSet
- The ReplicaSet creates Pods
- The scheduler assigns Pods to nodes
- kubelet on each node pulls the image and starts the container

Key parts

- replicas: 2
    - keep 2 copies of the app running
- image: andreiboboc123/app-demo:1.0
    - pull the container image from Docker Hub
- containerPort: 3000
    - application listens on port 3000 inside the container
- readinessProbe
    - tells Kubernetes when the app is ready to receive traffic
- livenessProbe
    - tells Kubernetes whether the app is still healthy

Why probes matter

Without health probes, Kubernetes would not know when to stop routing traffic to a broken or unready container.

Create the Service Manifest

vim service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: app-demo-service
spec:
  type: NodePort
  selector:
    app: app-demo
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30007
```

What it does

Exposes the application to traffic from outside the cluster.

Under the hood

- Kubernetes creates a Service object
- kube-proxy configures networking rules on each node
- traffic sent to NodeIP:30007 is forwarded to Pods matching label app: app-demo

Key parts

- type: NodePort
  - exposes the service externally through a port on each node
- selector
  - connects the Service to Pods with label app: app-demo
- port: 80
  - service port
- targetPort: 3000

    - container port
- nodePort: 30007
    - external port reachable from the host

##9. Apply the Manifests

kubectl apply -f ~/app-manifests/deployment.yaml

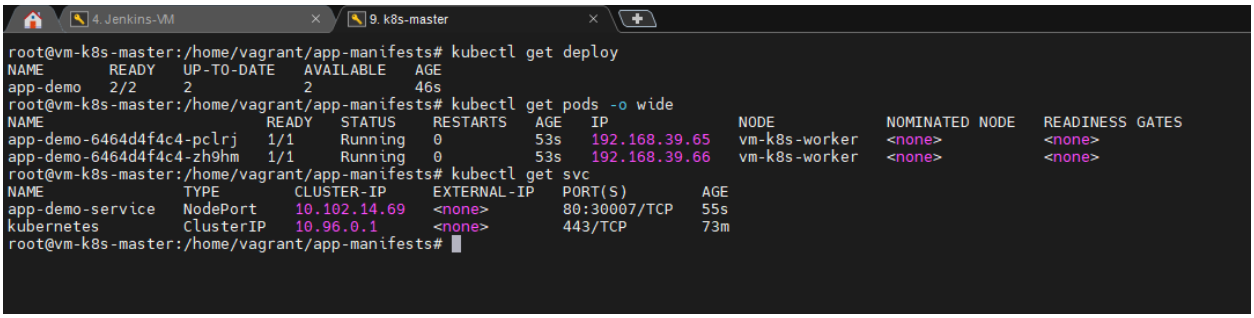
kubectl apply -f ~/app-manifests/service.yaml

##10. Verify Deployment and Service

kubectl get deployment

kubectl get pods -o wide

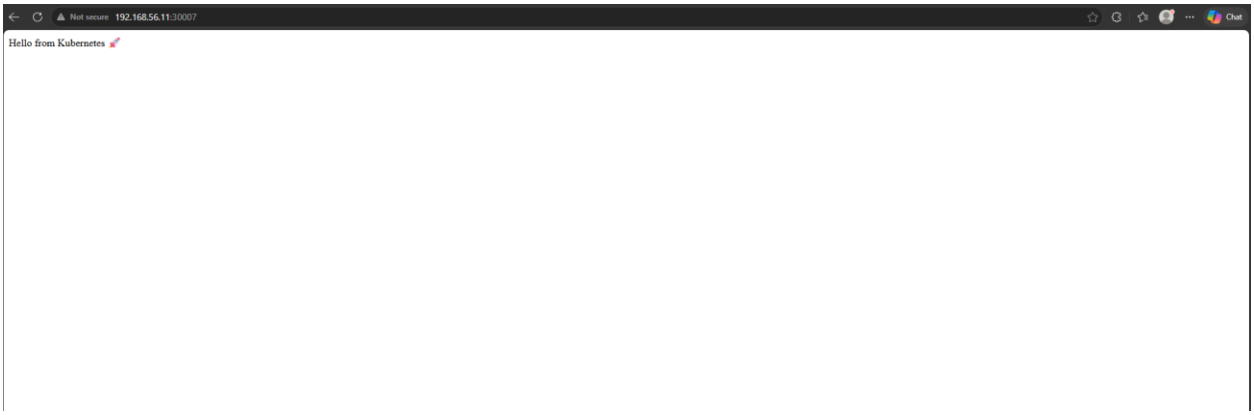
kubectl get svc



```
root@vm-k8s-master:/home/vagrant/app-manifests# kubectl get deploy
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
app-demo    2/2     2            2           46s
root@vm-k8s-master:/home/vagrant/app-manifests# kubectl get pods -o wide
NAME                        READY   STATUS    RESTARTS   AGE   IP               NODE            NOMINATED NODE   READINESS GATES
app-demo-6464d4f4c4-pclrj   1/1     Running   0          53s   192.168.39.65    vm-k8s-worker   <none>           <none>
app-demo-6464d4f4c4-zh9hm   1/1     Running   0          53s   192.168.39.66    vm-k8s-worker   <none>           <none>
root@vm-k8s-master:/home/vagrant/app-manifests# kubectl get svc
NAME                TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
app-demo-service    NodePort    10.102.14.69   <none>        80:30007/TCP   55s
kubernetes          ClusterIP   10.96.0.1      <none>        443/TCP        73m
root@vm-k8s-master:/home/vagrant/app-manifests#
```

##11. Access the application

http://192.168.56.11:30007



Under the hood

The browser request flows like this:

Browser
↓
Node IP:30007
↓
NodePort Service
↓
kube-proxy networking rules
↓
Selected Pod
↓
Node.js application on port 3000
↓
Response returned to browser